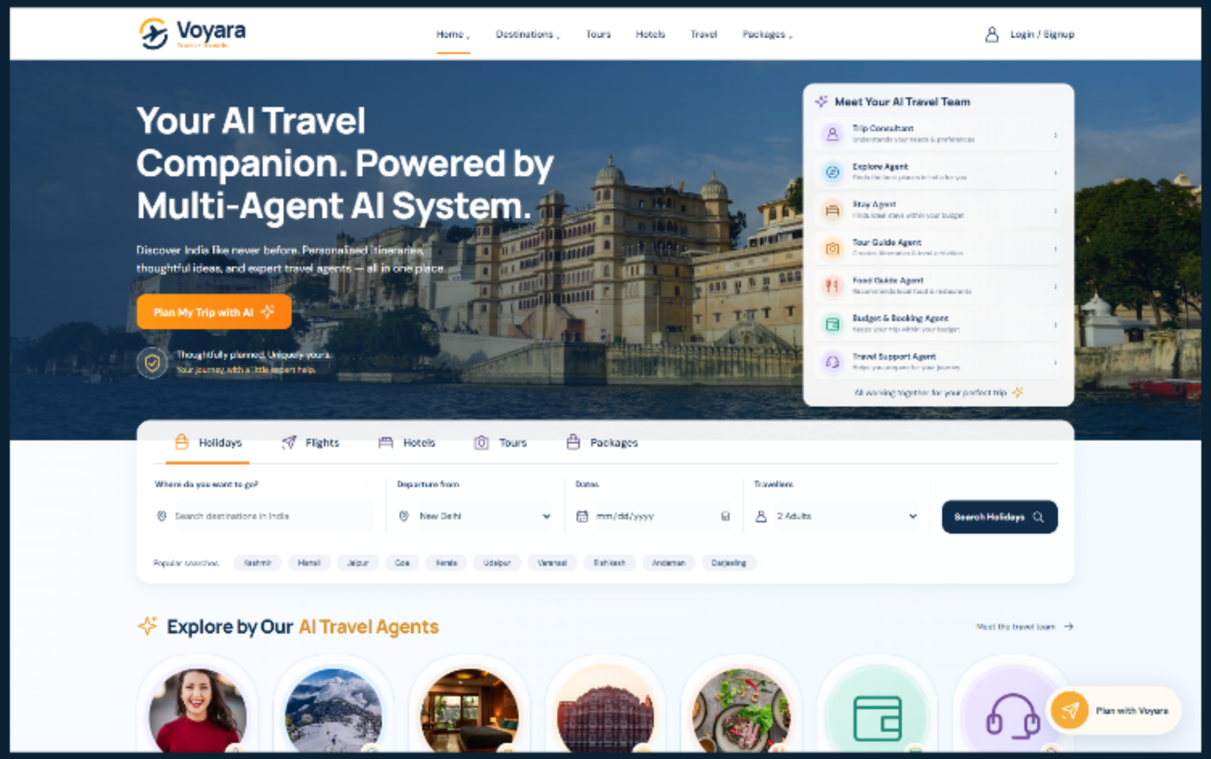
Iron Web + Voyara

AZINHACK ’26

Shared memory for travel planning agents

Current trip requirements, source evidence and visible agent execution



Team Unstoppable 4s

Leader Ishmeet Singh

Member Vansh Varshney

Member Aarush Srivastava

Travel plans change after the first prompt

The problem is keeping every specialist current

A traveler lowers the budget

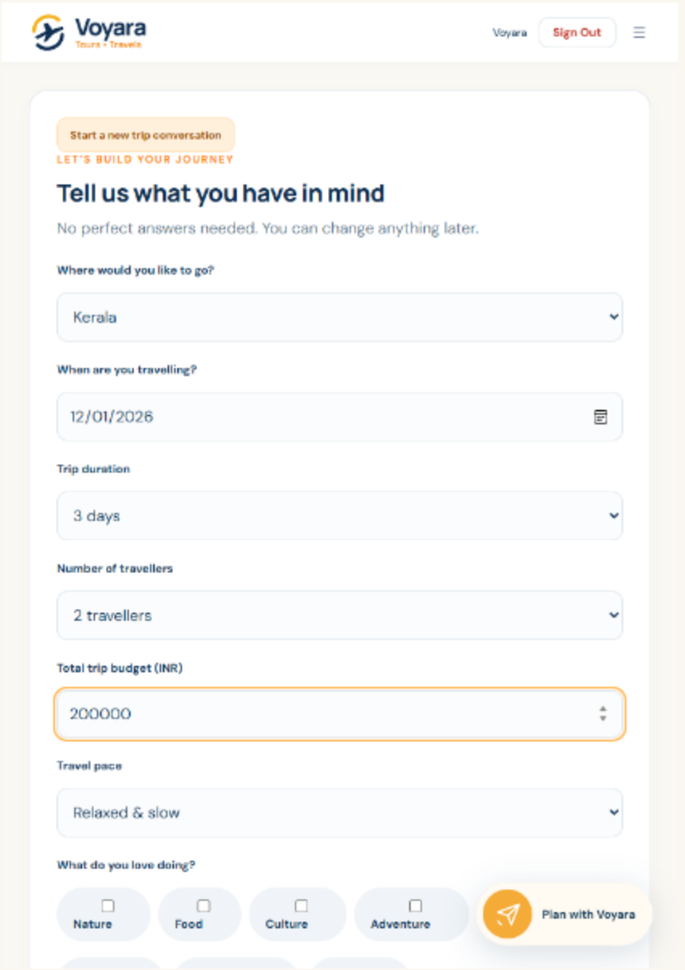
A cached plan can keep the old allowance unless the system tracks accepted changes.

Different specialists need the same facts

Stays, food and itinerary choices must use the same trip, dates and dietary requirements.

Voyara keeps the plan in one workflow

Travelers edit their trip details, review plan versions and export a private Travel Book PDF.



WHO IT HELPS

Travelers who revise plans

Teams that need traceable agent recommendations

Prototype impact: fewer repeated requirements and clearer plan revisions. User-outcome gains remain unmeasured.

System architecture and three-tier memory

One backend owns the connected memory runtime

Vercel website

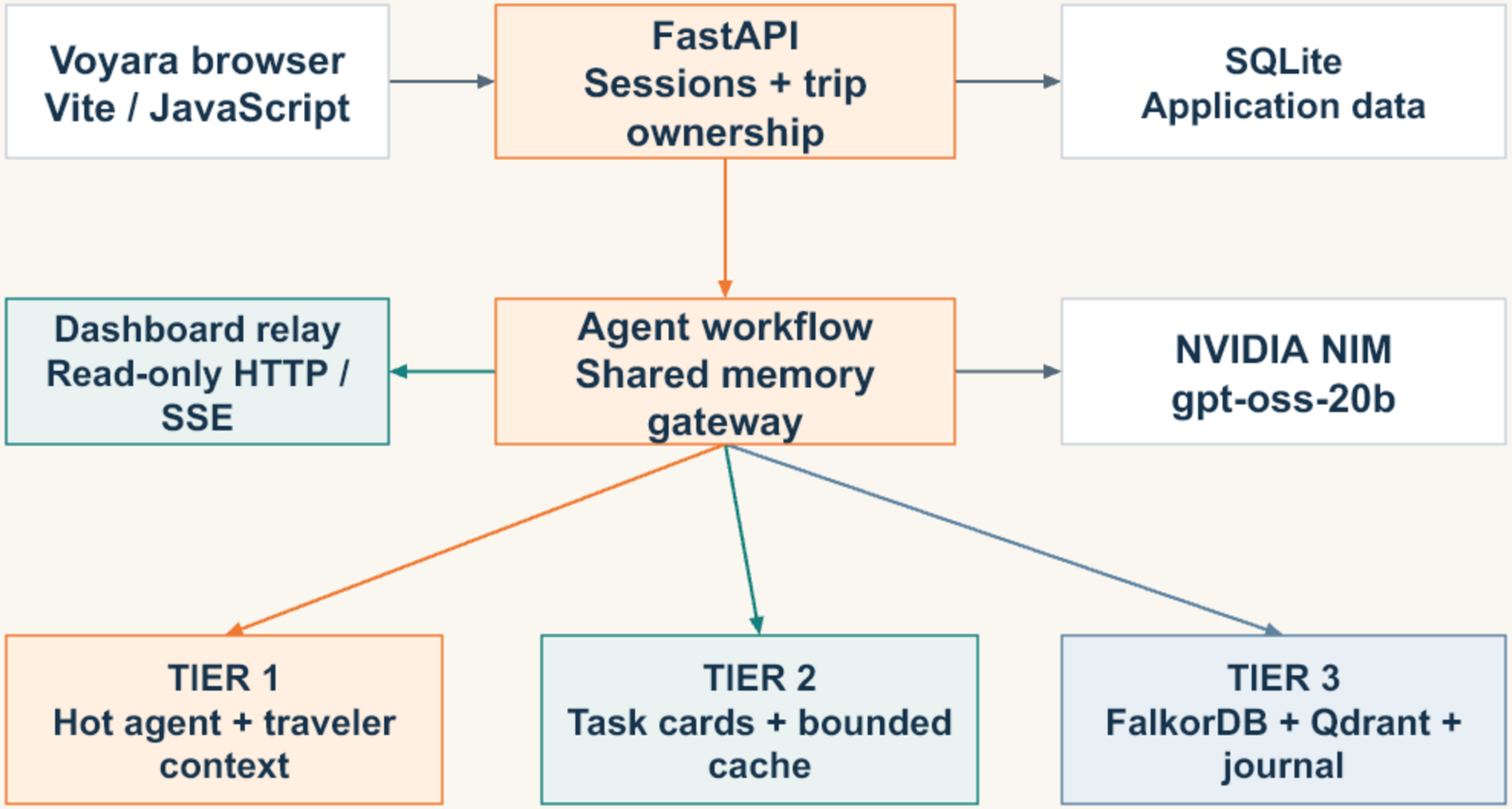
The frontend proxies /api to Render for same-origin sessions.

Render API + dashboard

FastAPI owns memory.
The relay feeds the dashboard.

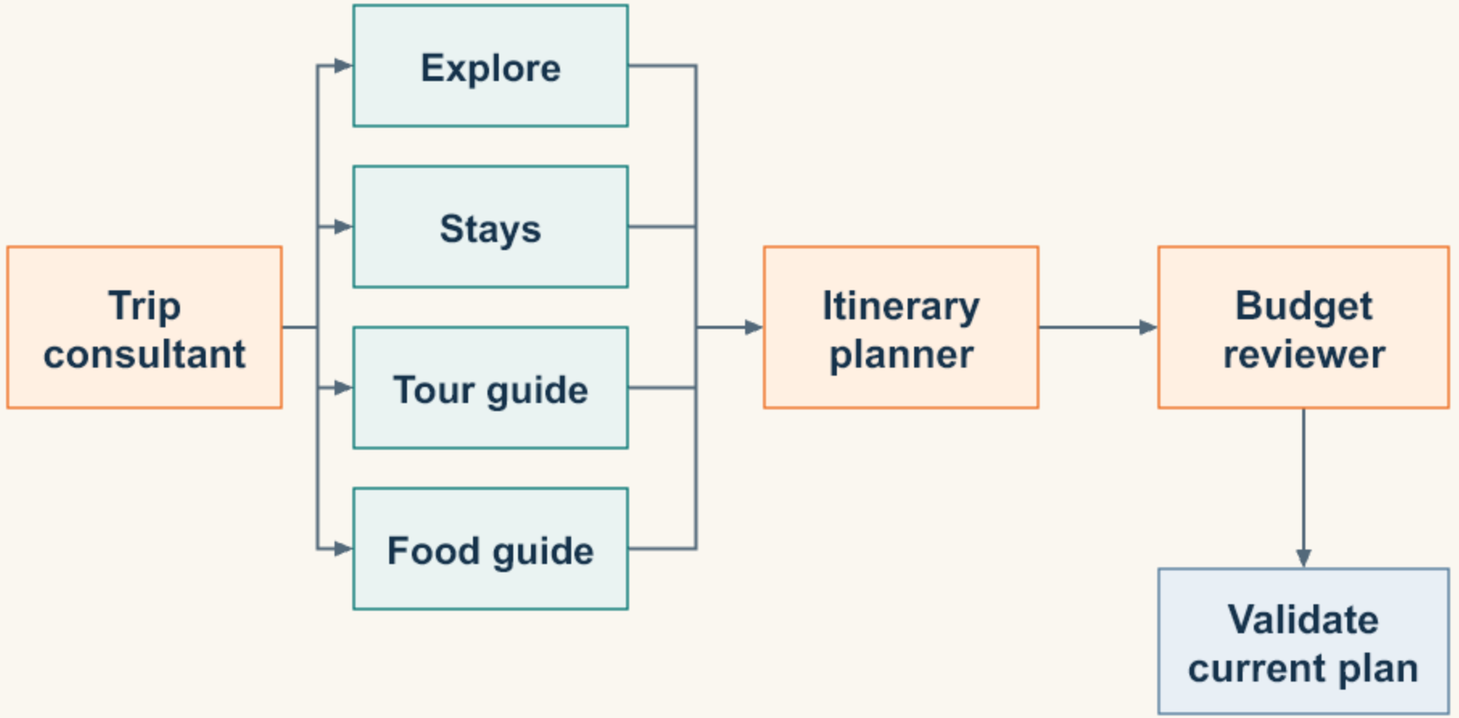


Recorded three-tier architecture view

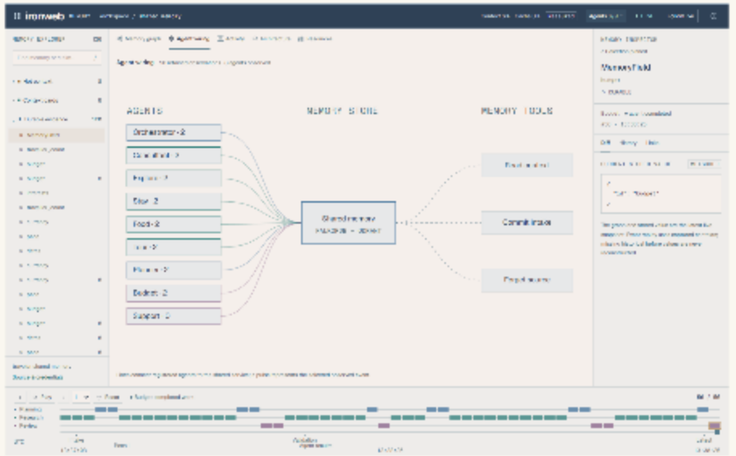


Seven specialists in the tested planning path

LangGraph routes independent research in parallel



The shared gateway supplies scoped evidence. Validation can request a bounded revision before the current plan returns.



Actual registered memory actors

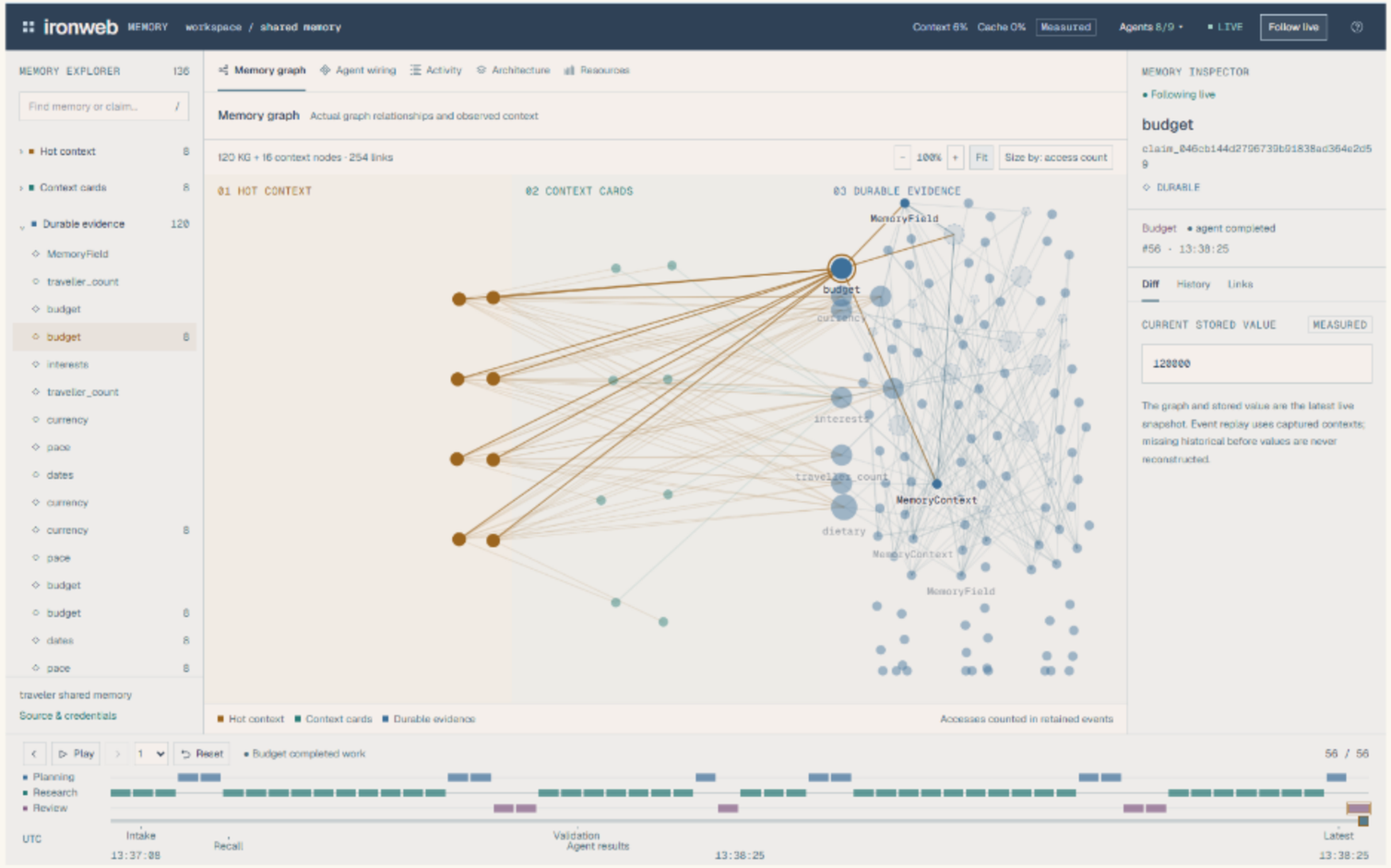
Typed tool actions

Schemas and allowlists constrain each specialist’s execution.

Only needed agents run

Accepted changes invalidate dependent work

A graph links each plan to the evidence it used



RECORDED BUDGET REVISION

INR 200,000
Original confirmed budget

↓

INR 120,000
Traveler accepts a new budget

↓

Review required
Old dependent work becomes stale

↓

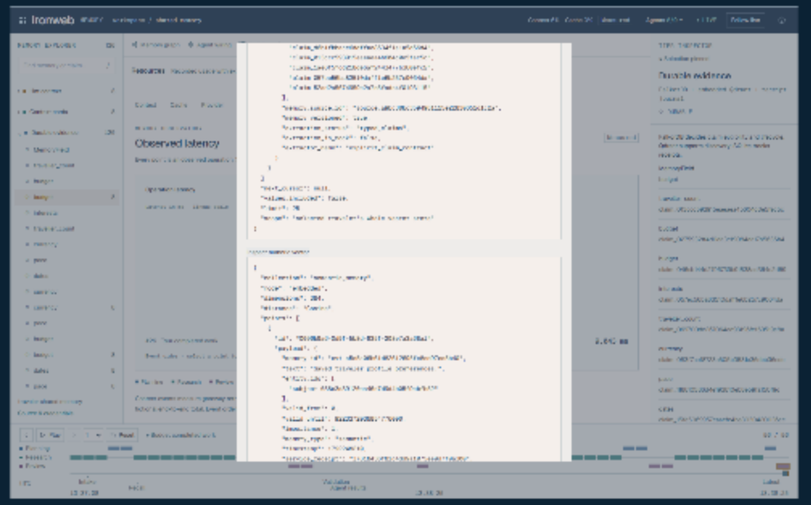
Current plan
Agents rebuild from eligible evidence

TinyFish integration for current web evidence

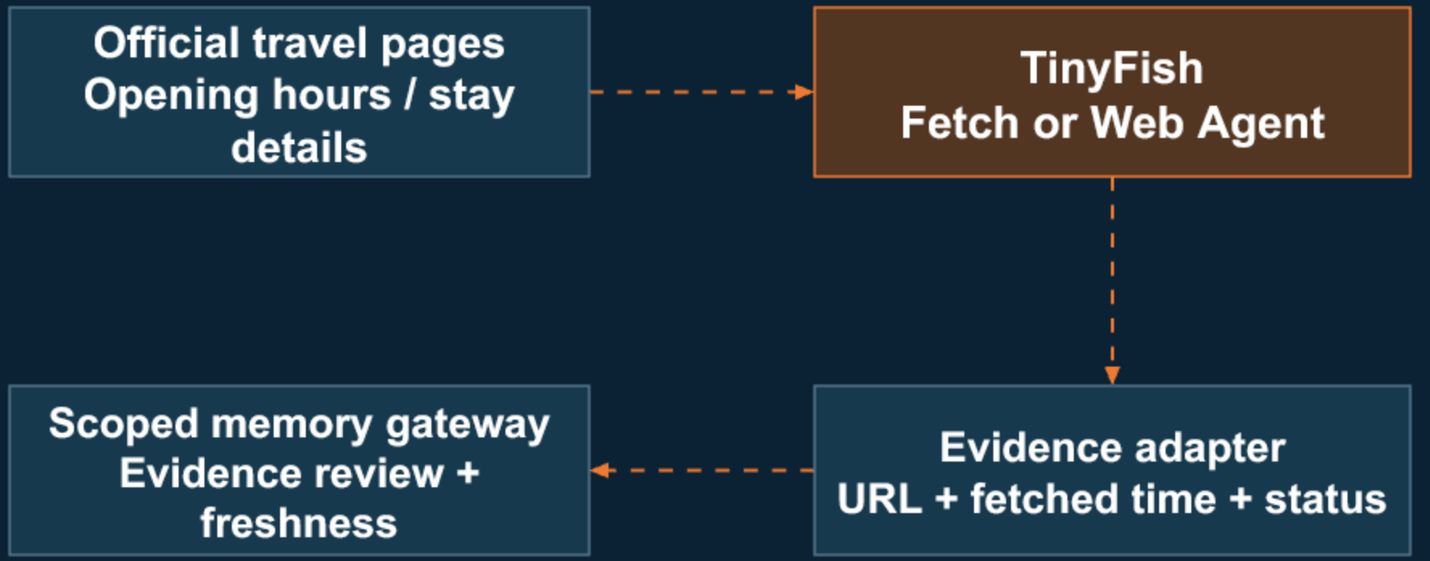
PLANNED INTEGRATION (NOT IMPLEMENTED)

Web facts with retained sources

TinyFish would fetch official travel pages. The adapter would retain URLs, fetch times and evidence status.



Existing source inspector; TinyFish remains planned.



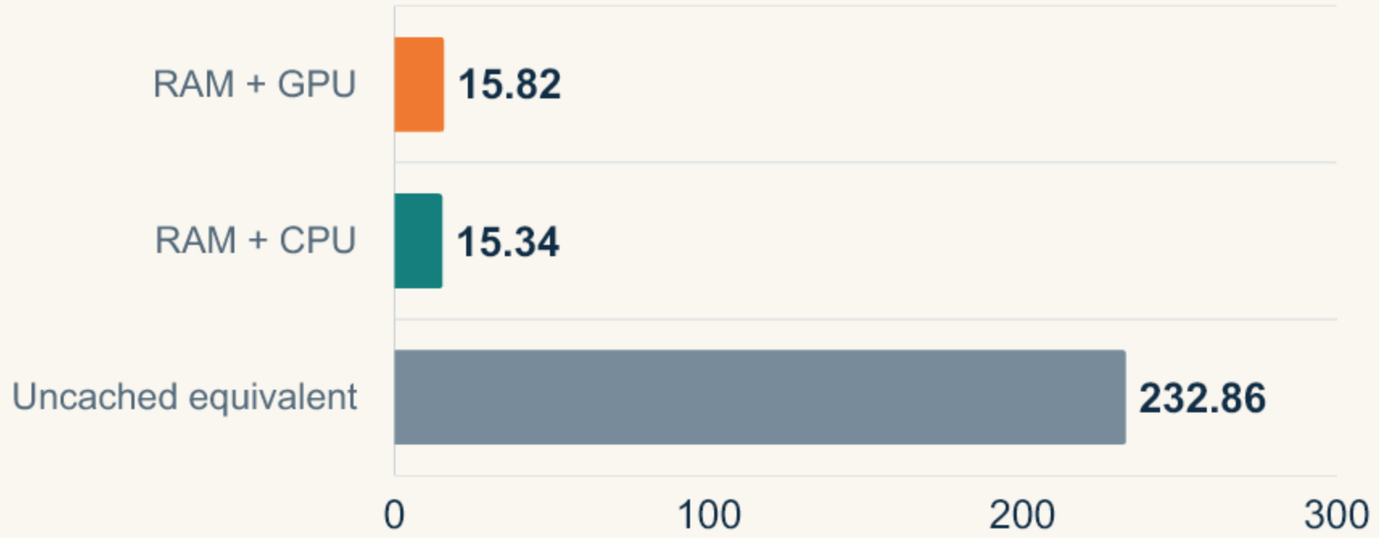
Acceptance before claiming integration

A real provider run, retained source evidence, error handling and a visible example in the README and demo video

Measured context preparation and correctness

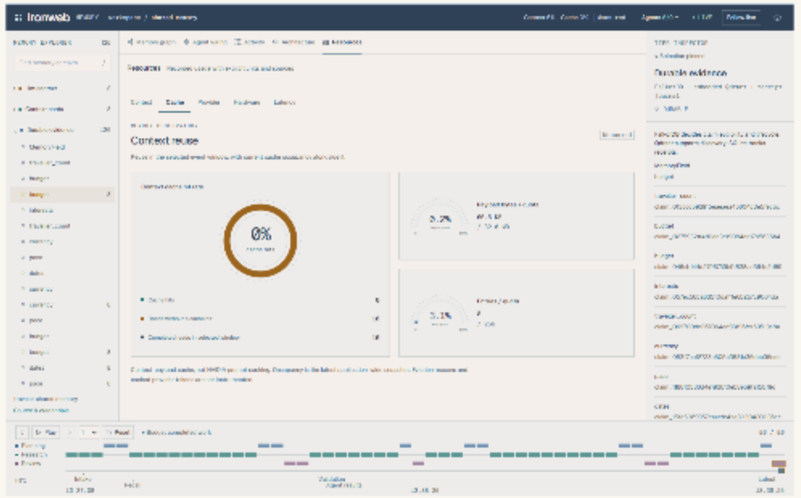
Warm caching reduced repeated context assembly

Median context preparation, milliseconds



15.2×

Lower median context assembly time with RAM + CPU in this fixture



Recorded cache view, separate from timed fixture

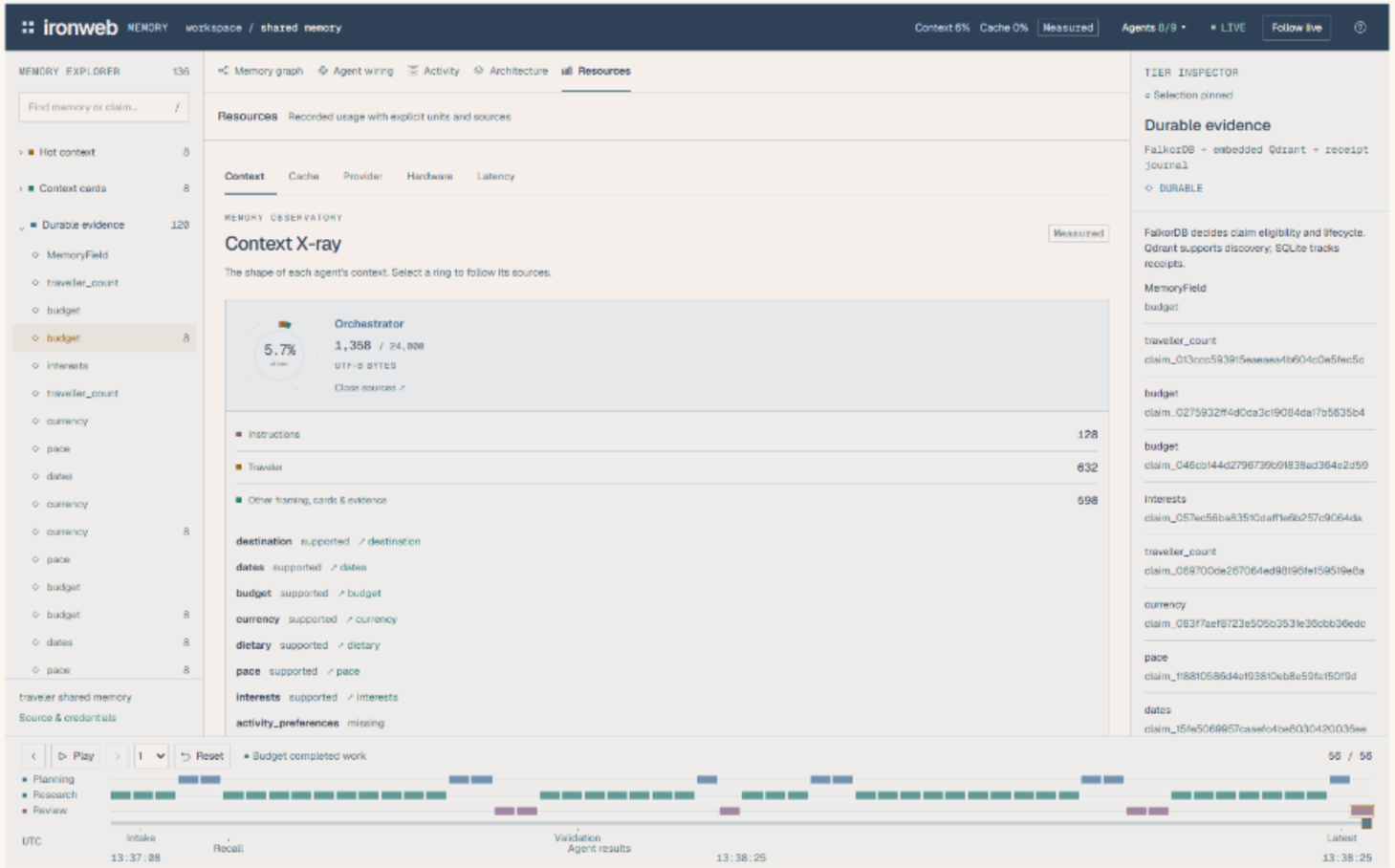
36/36 field checks correct

Supported, disputed and missing fields in a regression fixture

Local fixture: 15 repeats, one active card, identical prompt/evidence. Context preparation only; no generation timing.

Memory observability during execution

A read-only dashboard connects contexts, sources and agent events



Trace the selected context

Inspect the fields and source claims each agent used for its response.

Follow the actual work

Actor timelines link captured contexts to the selected event.



Recorded agent activity